COMPONENTE CURRICULAR:	Programação de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. Criando as Exceções

Para cada erro específico, uma classe.

1.1 Arquivo: NomeInvalidoException.java

```
NomeInvalidoException.java U ×
NomeInvalidoException.java > NomeInvalidoException
1  public class NomeInvalidoException extends Exception {
2      public NomeInvalidoException(String mensagem) {
3          super(mensagem);
4      }
5  }
```

1.2 Arquivo: PrecoInvalidoException.java

```
PrecoInvalidoException.java U ×
PrecoInvalidoException.java > PrecoInvalidoException
1  public class PrecoInvalidoException extends Exception {
2      public PrecoInvalidoException(String mensagem) {
3          super(mensagem);
4      }
5  }
```

1.3 Arquivo: QuantidadeInvalidaException.java

```java
QuantidadeInvalidaException.java U ×
QuantidadeInvalidaException.java > QuantidadeInvalidaException
public class QuantidadeInvalidaException extends Exception {
    public QuantidadeInvalidaException(String mensagem) {
        super(mensagem);
    }
}
```

2. Classe Produto

Este arquivo é o "molde". Ele agora vai conferir se os dados estão certos antes de aceitar.

```java
Produto.java U ×
Produto.java > Produto
public class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    public Produto(String nome, double preco, int quantidade)
            throws NomeInvalidoException, PrecoInvalidoException, QuantidadeInvalidaException {

        // Regra 1: Nome não pode ser vazio
        if (nome == null || nome.trim().isEmpty()) {
            throw new NomeInvalidoException(mensagem: "O nome não pode ser vazio.");
        }
        // Regra 2: Preço deve ser positivo
        if (preco <= 0) {
            throw new PrecoInvalidoException(mensagem: "O preço deve ser maior que zero.");
        }
        // Regra 3: Quantidade não pode ser negativa
        if (quantidade < 0) {
            throw new QuantidadeInvalidaException(mensagem: "A quantidade não pode ser negativa.");
        }

        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    @Override
    public String toString() {
        return "Produto: " + nome + " | Preço: R$" + preco + " | Quantidade: " + quantidade;
    }
}
```

3. O Gerenciador CadastroProdutos

Este arquivo guarda a lista de produtos e "tenta" adicionar novos, tratando os erros se eles aparecerem.

CadastroProdutos.java U

CadastroProdutos.java > CadastroProdutos

```java
import java.util.ArrayList;
import java.util.List;

public class CadastroProdutos {
    private List<Produto> produtos = new ArrayList<>();

    public void adicionarProduto(String nome, double preco, int quantidade) {
        try {
            // Tenta criar o produto. Se as regras do arquivo Produto.java falharem, pula pro 'catch'
            Produto novo = new Produto(nome, preco, quantidade);
            produtos.add(novo);
            System.out.println("SUCESSO: " + novo);
        } catch (NomeInvalidoException | PrecoInvalidoException | QuantidadeInvalidaException e) {
            // Mostra a mensagem de erro que definimos lá no Produto.java
            System.out.println("ERRO AO CADASTRAR: " + e.getMessage());
        }
    }

    public void listarProdutos() {
        System.out.println("\n--- PRODUTOS NO ESTOQUE ---");
        for (Produto p : produtos) {
            System.out.println(p);
        }
    }
}
```

4. A classe Principal

Arquivo que vai executar o programa.

Principal.java U

Principal.java > Principal

```java
public class Principal {
    Run | Debug
    public static void main(String[] args) {
        CadastroProdutos cadastro = new CadastroProdutos();

        // Teste 1: Tudo certo
        cadastro.adicionarProduto(nome: "Celular", preco: 9500.0, quantidade: 17);

        // Teste 2: Nome vazio (Vai dar erro)
        cadastro.adicionarProduto(nome: "", preco: 249.99, quantidade: 15);

        // Teste 3: Preço negativo (Vai dar erro)
        cadastro.adicionarProduto(nome: "Fone Bluetooth", -55.90, quantidade: 20);

        // Teste 4: Quantidade negativa (Vai dar erro)
        cadastro.adicionarProduto(nome: "Teclado", preco: 150.0, -3);

        // Mostra apenas o que funcionou
        cadastro.listarProdutos();
    }
}
```

5. Terminal

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Run: Principal

PS C:\Users\murillo.silva\Documents\GitHub\ads\semestre-3\programacao-de-sistemas-ii\A2-praticando\TratamentoDeExececoes>  & 'C:\Program Files\Java\jre
1.8.0_421\bin\java.exe' '-cp' 'C:\Users\murillo.silva\AppData\Roaming\Code\User\workspaceStorage\e4806b81f90fa28f442f15033b30da40\redhat.java\jdt_ws\Tr
atamentoDeExececoes_d1b5619c\bin' 'Principal'
SUCESSO: Produto: Celular | Preço: R$9500.0 | Quantidade: 17
ERRO AO CADASTRAR: O nome não pode ser vazio.
ERRO AO CADASTRAR: O preço deve ser maior que zero.
ERRO AO CADASTRAR: A quantidade não pode ser negativa.

--- PRODUTOS NO ESTOQUE ---
Produto: Celular | Preço: R$9500.0 | Quantidade: 17
```